

1. CANONICAL ENTITY DESIGN RATIONALE

I designed the canonical payment entity around the fields shared by all three Mal product squads: identity, monetary attributes, lifecycle status, customer ownership, and operational context. The core of the model is intentionally flat so Cards, Transfers, Bill Payments, Finance, Risk, and CRM teams can all query one table without first learning three squad-specific schemas. Product-specific details such as `merchant_name`, `counterparty_id`, and `biller_code` remain nullable extension fields inside the same contract rather than being pushed into separate subtype tables. That makes the first rollout materially easier because downstream consumers get one reusable grain: one row per payment event.

The schema also leaves room for future payment types such as QR merchant collections, salary disbursements, or Islamic financing repayments. Extensibility comes from three choices: `payment_type` is controlled but expandable, optional fields support partial specialization, and contract versioning separates source change timing from platform change timing. For Mal, this matters because new UAE payment rails or partner billers will likely arrive faster than every squad can refactor at once.

The main trade-off is simplicity over full normalization. I did not split the canonical entity into separate payment, party, merchant, and status-history tables because that would slow adoption for an assessment-sized first release. The flat model duplicates some context and cannot represent every lifecycle transition in detail, but it is easier to validate, easier to query, and much easier to roll out across three independent squads. That is the right first platform move.

2. PHASED MIGRATION PLAN

First 30 days

Stand up the shared contract, reference mappings, and validation rules under a platform-owned repository. Pair with each squad lead to document the current Cards, Transfers, and Bill Payments fields, required semantics, and known data quality issues. Publish the canonical v1 contract, sample records, SQL examples, and a CI-friendly validation harness. Sequence adoption by starting with Bill Payments first, then Cards, then Transfers.

Bill Payments is the best first adopter because it is usually lower volume, structurally simpler, and has fewer real-time fraud dependencies than cards or outbound transfers. That lets the platform team prove the model and the rollout process with lower operational risk. During this phase, squads keep their existing pipelines unchanged while the platform layer ingests CSV or service exports and produces the unified table in parallel.

Days 31-60

Move Cards onto the shared contract next. Cards adds the most visible merchant analytics value and exposes cross-border behavior through `country_code`, so it is the best second

milestone for demonstrating reuse value to stakeholders. Introduce v2 of the contract with channel and country_code, but keep the migration backward compatible by allowing source systems to continue emitting v1 while the platform migrates records during ingestion.

At the same time, define dependency ownership explicitly. The platform team owns the canonical contract, validation library, and publish step. Each squad owns source extraction quality and field mapping correctness. Analytics engineering or data consumers own semantic marts derived from the canonical table, not direct schema decisions.

Days 61-90

Bring Transfers onto the shared path last because outbound transfers typically carry the highest controls burden: sanctions screening context, beneficiary semantics, and cross-border reporting needs. By this point, the other two squads provide proof that the shared model is stable and useful. Replace ad hoc squad extracts with a scheduled ingestion job, attach data quality SLAs, and require contract tests for any schema-affecting change. Transition downstream reporting to the unified table and deprecate duplicate squad-specific reporting tables after a fixed overlap window.

Backward compatibility during the whole 90-day period comes from three rules: only additive changes are allowed inside a major version, migration code is owned centrally, and the platform publishes change notices before removing old fields or versions. If a squad cannot move immediately, the platform layer continues accepting the previous version until the agreed deprecation date.

3. DATA CONTRACT & GOVERNANCE

Breaking changes require a major version bump such as v1 to v2; non-breaking additions stay within the same major version. Every contract version must include a changelog entry, sample payload, validation rules, and an owner. Validation should be enforced in three places: at source mapping tests inside each squad repo, at ingestion time before publish, and in warehouse or serving-layer quality monitors after load.

Ownership should be explicit. The data platform team maintains the canonical contract, migration code, and release process. Each product squad nominates a data owner who approves mapping changes from their domain. Risk and Finance should be required reviewers for any changes that affect payment status semantics, amount fields, or cross-border attributes, because those changes impact regulatory and reconciliation use cases in a UAE banking context.

4. ADOPTION METRICS & STAKEHOLDER PLAN

The shared-platform program should be measured with a small KPI set tied to behavior, not just delivery.

1. Canonical coverage rate: percent of payment volume landing through the unified contract.
2. Reuse rate: number of downstream dashboards or jobs migrated off squad-specific tables.
3. Schema compliance rate: percent of records that pass validation without manual

intervention.

4. Time-to-onboard a new payment source: days from field mapping to first successful publish.

5. Incident rate tied to payment data mismatches or delayed loads.

Communication has to be practical and squad-oriented. I would run a fortnightly contract review with Cards, Transfers, Bill Payments, Risk, and Analytics representatives; publish a one-page release note for every contract change; and maintain a shared Slack channel with example queries, rollout status, and known issues. To handle resistance, I would avoid selling ?standardization? in the abstract. I would show each squad what they get back: fewer one-off data requests, faster analytics support, shared controls for cross-border monitoring, and reduced duplication when new features launch across products.

For squads with entrenched pipelines, the platform team should lower the switching cost. Provide the field mapping template, the test harness, sample outputs, and migration support rather than forcing teams to redesign everything first. Adoption is much faster when the platform absorbs most of the integration burden.

5. PRODUCTION CONSIDERATIONS

At 100K transactions per day, I would replace local-file ingestion with object storage landing zones, orchestrate runs with Airflow or Dagster, and publish into a warehouse table format such as Delta or Iceberg. The validation layer would become both a CI contract test and a runtime quality gate. I would also add idempotency keys, partitioning by event date, structured logging, and dead-letter handling for rejected rows.

Monitoring would cover pipeline freshness, input/output row counts, rejection rates by source, major field null rates, and reconciliation checks between source counts and published counts. Alerting should route differently based on severity: data engineering for freshness and runtime failures, product squads for mapping regressions, and Finance or Risk stakeholders for reconciliation or cross-border data defects.

What I intentionally cut from Part 1 was real orchestration, cloud deployment, secrets handling, and a full semantic warehouse model. I also did not model refunds, reversals, or multi-step payment lifecycle events in detail. Those omissions were deliberate so the submission could stay small, runnable locally, and focused on the most important platform behaviors: canonical modeling, controlled migration, validation, and adoption strategy.